

《计算机视觉》实验报告

姓 名： 黄俊霖

学 号： 202308010523

湖南大学信息科学与工程学院

2026 年春季学期

实验一 人像分割实验

1. 实验流程

- a) 搭建实验环境，安装 torch、torchvision、opencv-python、numpy 等依赖。

```
≡ requirements.txt
1  opencv-python>=4.8.0
2  torch>=2.0.0
3  torchvision>=0.15.0
4  numpy>=1.24.0
5
```

- b) 准备测试图像（如 people.jpg）。

```
def build_parser():
    parser = argparse.ArgumentParser(description="Simple portrait segmentation")
    parser.add_argument("input", help="Input image path")
    parser.add_argument(
        "-o",
        "--output",
        help="Output image path (default: <input_name>_portrait.png)",
    )
    return parser
```

- c) 使用 DeepLabV3 预训练模型（COCO）进行人像语义分割，提取人物类别掩码（class id=15）。

```
# 使用预训练模型（在 COCO 上训练）
self.model = segmentation.deeplabv3_resnet50(pretrained=True)
self.model.eval()
self.model.to(self.device)
```

```
# 获取分割结果
output = output['out']
pred = torch.argmax(output, dim=1).squeeze().cpu().numpy()

# COCO 数据集中，人物类别是 15
# 其他类别可参考：0=背景，15=人
person_mask = (pred == 15).astype(np.uint8) * 255
```

- d) 对分割掩码进行后处理（形态学操作）以减少噪点、填补小空洞。

```
# 形态学处理, 去除小噪点
kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
person_mask = cv2.morphologyEx(person_mask, cv2.MORPH_CLOSE, kernel)
person_mask = cv2.morphologyEx(person_mask, cv2.MORPH_OPEN, kernel)
```

- e) 将掩码作为 Alpha 通道, 生成透明背景人像图 (PNG)。

```
# 转换为 RGBA
if image.shape[2] == 3:
    result = cv2.cvtColor(image, cv2.COLOR_BGR2BGRA)
else:
    result = image.copy()

# 设置 alpha 通道
result[:, :, 3] = mask
```

- f) 通过命令行运行程序: 输入原图路径, 输出透明背景结果图。

2. 实验结果

输入: people.jpg

输出: people_portrait.png (透明背景)

程序可稳定完成单图分割, 输出图像能够保留主体人像并去除大部分背景。

3. 实验结果分析

对基本实验和优化后实验结果进行对比分析:

基本实验: 直接使用模型输出掩码, 边缘存在一定锯齿, 局部区域 (如发丝、衣物边界) 会有少量误分割。

优化后实验: 加入掩码后处理 (形态学开闭处理) 并输出 Alpha 通道透明图后, 主体边界更连贯, 背景残留明显减少, 整体视觉效果更自然。

思考:

预训练模型在通用场景表现较好, 但对复杂背景、细小边缘仍有提升空间;

后处理能显著改善可视化效果, 但参数过强会导致边缘细节损失;

后续若有标注数据, 可做微调提升特定场景精度。。

4 实验总结

4.1 实验中遇到的问题与解决办法

环境依赖问题: 部分 Python 环境下动态库不兼容, 导致 pip/依赖安装异常。

解决: 切换到稳定 Python 3.11 虚拟环境, 修复动态库依赖后重新安装。

模型首次加载较慢: 首次运行需下载模型权重。

解决: 提前联网下载并缓存权重, 后续运行速度明显提升。

边缘噪点问题：原始掩码存在噪点和小孔洞。

解决：加入形态学后处理，提升分割质量。

4.2 实验收获与不足

收获：掌握了语义分割完整流程（预处理→推理→后处理→结果导出），并理解了模型输出与可视化质量之间的关系。

不足：当前主要是单图处理与定性评估，缺少标准数据集上的定量指标（如 IoU、mIoU）对比。

4.3 其它感想（欢迎吐槽）

这次实验最“真实”的部分不是模型推理，而是环境折腾。模型跑起来后，图像效果提升非常直观，成就感很强；但也能明显感觉到，工程问题（环境、依赖、兼容性）有时候比算法本身更耗时间。

实验二 车辆计数实验

1. 实验流程

- a) 搭建环境并安装依赖（ultralytics、opencv-python、numpy）。

```
requirements.txt
1  ultralytics>=8.0.0
2  opencv-python>=4.8.0
3  numpy>=1.24.0
4  lap>=0.5.12
```

- b) 准备测试视频 vehicle.mp4。

```
6  BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
7  INPUT_VIDEO = os.path.join(BASE_DIR, "vehicle.mp4")
8  OUTPUT_DIR = os.path.join(BASE_DIR, "output_videos")
9  OUTPUT_VIDEO = os.path.join(OUTPUT_DIR, "output_counted.mp4")
10 MODEL_PATH = "yolov8n.pt" # 首次运行自动下载
```

- c) 使用 YOLOv8 进行车辆检测，结合 ByteTrack 进行跨帧 ID 跟踪。

```
def __init__(self):
    self.device = get_device()
    print(f"[Tracker] 使用设备: {self.device}")
    self.model = YOLO(MODEL_PATH)

def update(self, frame):
    """
    对单帧执行检测 + 追踪。

    Args:
        frame: BGR numpy array (OpenCV 帧)

    Returns:
        ultralytics Results 对象 (boxes 含 track_id)
        若无结果返回 None。
    """
    results = self.model.track(
        source=frame,
        conf=CONF_THRESHOLD,
        iou=IOU_THRESHOLD,
        classes=VEHICLE_CLASSES,
        tracker=TRACKER,
        device=self.device,
        persist=True,  # 跨帧持久化追踪状态 (关键!)
        verbose=False,
    )
    return results[0] if results else None
```

- d) 设置虚拟计数线，基于目标中心点跨线规则进行计数。

```
# — 计算计数线绝对坐标 —
line_y = int(height * config.COUNTING_LINE_Y_RATIO)
print(f"[计数线] y = {line_y}px (帧高 {config.COUNTING_LINE_Y_RATIO * 100:.0f}%)")
```

```
for tid, box in zip(ids, xyxy):
    cy = (box[1] + box[3]) / 2.0  # 当前帧中心点 y

    if tid in self._prev_y:
        prev_cy = self._prev_y[tid]
        # 方向向上: 当前帧 y < 上一帧 y (车辆从画面下方向上方移动)
        if cy < prev_cy:
            self.upward_ids.add(tid)
            # 穿越检测: 上行 = y 减小方向穿越计数线
            if cy <= self.line_y < prev_cy and tid not in self._cooldown:
                self.count += 1
                self._cooldown[tid] = COOLDOWN_FRAMES
            # 第一次出现的 tid 不记入 upward_ids, 先记录位置, 下一帧再判断

    self._prev_y[tid] = cy

return self.count
```

- e) 对每帧叠加可视化信息 (检测框、Track ID、计数看板、计数线)。

```
# 1. 绘制计数线
cv2.line(
    out,
    (0, self.line_y),
    (self.frame_width, self.line_y),
    LINE_COLOR,
    THICKNESS,
)
```

```

# 2. 绘制车辆边框 + Track ID
if tracks is not None and tracks.bboxes is not None:
    boxes = tracks.bboxes
    xxyy = boxes.xxyy.cpu().numpy()
    ids = boxes.id.cpu().numpy().astype(int) if boxes.id is not None else []

    for i, box in enumerate(xxyy):
        # 若指定了 upward_ids, 只绘制上行车辆; 否则绘制所有 (向后兼容)
        if len(ids) > i:
            tid = ids[i]
            if upward_ids is not None and tid not in upward_ids:
                continue
        else:
            tid = None
            if upward_ids is not None:
                continue # 无 ID 时若启用过滤则跳过

        x1, y1, x2, y2 = map(int, box)
        cv2.rectangle(out, (x1, y1), (x2, y2), BOX_COLOR, THICKNESS)

        if tid is not None:
            label = f"ID:{tid}"
            (tw, th), _ = cv2.getTextSize(
                label, cv2.FONT_HERSHEY_SIMPLEX, FONT_SCALE * 0.6, 1
            )
            cv2.rectangle(
                out,
                (x1, y1 - th - 6),
                (x1 + tw + 4, y1),
                BOX_COLOR, -1,
            )
            cv2.putText(
                out, label,
                (x1 + 2, y1 - 4),
                cv2.FONT_HERSHEY_SIMPLEX,
                FONT_SCALE * 0.6,
                TEXT_COLOR, 1, cv2.LINE_AA,
            )

# 3. 绘制计数看板 (左上角半透明背景)
panel_text = f"Count: {count}"
(pw, ph), _ = cv2.getTextSize(
    panel_text, cv2.FONT_HERSHEY_SIMPLEX, FONT_SCALE, THICKNESS
)
overlay = out.copy()
cv2.rectangle(overlay, (10, 10), (10 + pw + 20, 10 + ph + 20), (0, 0, 0), -1)
cv2.addWeighted(overlay, 0.5, out, 0.5, 0, out)
cv2.putText(
    out, panel_text,
    (20, 10 + ph + 5),
    cv2.FONT_HERSHEY_SIMPLEX,
    FONT_SCALE,
    TEXT_COLOR, THICKNESS, cv2.LINE_AA,
)

```

f) 将处理后视频输出到 output_videos/output_counted.mp4。

2. 实验结果

输入视频总帧数：984 帧

程序处理完成后累计计数：54

输出文件：output_videos/output_counted.mp4（及测试输出 smoke.mp4）

输出视频可正常播放，检测框、ID 和计数信息显示完整。

3. 实验结果分析

对基本实验和优化后实验结果进行对比分析：

基本实验（仅检测或无稳定跟踪）：

同一车辆在多帧中可能被重复计数；

遮挡或短时丢检时计数波动较大。

优化后实验（YOLOv8 + ByteTrack + 防抖冷却）：

借助 `track id` 维持跨帧一致性，重复计数明显减少；

冷却帧机制降低了边界抖动导致的误触发；

计数稳定性和可解释性明显提升。

思考：

计数规则对方向判定非常敏感（坐标系方向必须明确）；

工程细节（编码器兼容、视频写出策略）直接影响最终可用性；

后续可引入多线/双向统计和精度评估。

4 实验总结

4.1 实验中遇到的问题与解决办法

计数方向判断易写反：上行车辆在图像中 `y` 是减小而非增大。

解决：使用 `cy <= line_y < prev_cy` 作为上行跨线条件。

慢放实现不稳定：单纯修改 `FPS` 在部分播放器下无效。

解决：每帧重复写入两次，实现稳定的 0.5x 慢放效果。

视频编码兼容问题：不同平台对编码器支持不同。

解决：实现 `avc1 -> mp4v -> MJPG` 回退链，提高可用性。

4.2 实验收获与不足

收获：掌握了“检测+跟踪+计数+可视化+导出”的完整视频分析链路，理解了跟踪在计数任务中的关键价值。

不足：目前主要是单场景测试，缺少多场景、多时段视频的鲁棒性验证；计数精度尚未进行人工标注对齐评估。

4.3 其它感想（欢迎吐槽）

这个实验最大的感受是：算法选型很重要，但“把结果稳定地输出成一个能看、能用、能复现的视频”更考验工程能力。

一句话总结：写模型是开始，做成可交付结果才是实验真正完成。